Università degli Studi di Catania

Corso di Laurea in Informatica Magistrale

Relazione Finale

Mixed Reality and Wearable Vision

EchoLens: la realtà aumentata che dà voce allo spazio

Un'applicazione per Oculus Meta Quest 3 a supporto degli ipovedenti

Studente: Alessia Micieli

Matricola: 1000030171

Supervisori: Professor Giovanni Maria Farinella e Dottor Michele Mazzammuto

Anno Accademico 2024/2025

Indice

1 Introduzione **3**

2 Stato dell'arte **5**

2.1 Dispositivi "on-eye": visione artificiale direttamente sugli occhiali 5

2.2 Servizi di interpretariato visivo in remoto 6

2.3 Mixed Reality con pass-through video 6

2.4 EchoLens 7

3 Sviluppo dell'applicazione **8**

3.1 Struttura della scena in Unity 8

3.2 Architettura di Comunicazione e Streaming Real-Time 11

3.2.1 Scoperta Automatica del Server 11

3.2.2 Streaming dei Frame Video via WebSocket 11

3.2.3 Elaborazione dei Frame sul Server Python 12

3.2.4 Flusso Completo di Comunicazione 12

3.2.5 Caratteristiche e Vantaggi dell'Architettura 13

3.3 Ancore Spaziali 14

3.3.1 Preparazione dell'ambiente Unity 14

3.3.2 Gestione del pinch 14

3.3.3 Creazione e mantenimento degli oggetti virtuali 15

3.3.4 Risultati 15

3.3.5 Creazione e mantenimento degli oggetti virtuali 16

3.3.6 Risultati 16

4 Conclusioni **17**

Capitolo 1

Introduzione

Negli ultimi anni, i dispositivi di realtà mista e i visori di realtà aumentata e virtuale hanno trovato applicazioni in ambiti che vanno oltre l'intrattenimento, interessando settori come la medicina, l'educazione e l'assistenza.

In questo contesto, il progetto presentato in questa relazione si concentra sull'utilizzo del visore *Oculus Meta Quest 3* per sviluppare un'applicazione rivolta a persone ipovedenti.

L'obiettivo principale è fornire un supporto innovativo alla percezione dell'ambiente circostante, permettendo agli utenti di ricevere una descrizione della scena in tempo reale attraverso lo streaming video acquisito dal visore. Questa soluzione mira a migliorare l'autonomia degli ipovedenti, offrendo un sistema di assistenza indossabile che combina realtà mista, computer vision e tecniche di intelligenza artificiale.

La relazione è strutturata come segue:

- Nel Capitolo 2 viene analizzato il contesto tecnologico e lo stato dell'arte, con particolare attenzione ai dispositivi wearable e alle applicazioni rivolte all'accessibilità.

- Nel Capitolo 3 viene descritto lo sviluppo dell'applicazione, dagli strumenti utilizzati alle scelte progettuali.

- Nel Capitolo 4 vengono discussi i risultati ottenuti e le possibili applicazioni future.

- Infine, nel Capitolo 5 vengono riportate le conclusioni e le prospettive di miglioramento del progetto.

Capitolo 2

Stato dell'arte

Negli ultimi anni le tecnologie indossabili e la *Mixed Reality* (MR) hanno abilitato nuove soluzioni di accessibilità per persone ipovedenti o cieche. In questo capitolo si sintetizzano i principali filoni: (i) dispositivi indossabili di *computer vision* "on-eye", (ii) servizi di interpretariato visivo in remoto, (iii) applicazioni MR con pass-through video su visori di nuova generazione.

2.1 Dispositivi "on-eye": visione artificiale direttamente sugli occhiali

OrCam MyEye

OrCam MyEye è un dispositivo commerciale che si aggancia magneticamente alla montatura degli occhiali e fornisce feedback audio in tempo reale. Il sistema legge testi, riconosce oggetti e volti, banconote e prodotti; tra le modalità di attivazione vi è anche il *gesture* di puntamento con il dito, oltre al tap laterale del modulo[1]. Questo approccio ha mostrato la maturità del mercato verso soluzioni miniaturizzate, *always-on* e a mani libere, oggi spesso

[1] Vedi *OrCam MyEye 2 Pro* (pagina prodotto): https://www.orcam.com/en-us/orcam-myeye-2-pro.

arricchite da modelli di riconoscimento on-device e da pipeline ibride edge–cloud.[2]

Envision Glasses

Un'altra soluzione *state-of-the-practice* sono le *Envision Glasses*, basate su *Google Glass Enterprise Edition 2*, che offrono lettura di testo, descrizione di scena, rilevazione di colori e luce, riconoscimento di volti insegnati dall'utente, ricerca di oggetti, oltre a chiamate a un contatto o a un interprete Aira[3].

2.2 Servizi di interpretariato visivo in remoto

Due servizi diffusi sono *Aira* e *Be My Eyes*. Aira connette l'utente con interpreti professionisti che forniscono, via audio, descrizioni e guida in tempo reale a partire dallo stream video dello smartphone o di *wearables*[4]. *Be My Eyes* ha introdotto la componente *Be My AI*, che sfrutta modelli multimodali per descrivere immagini inviate dall'utente e rispondere a domande sul contenuto visivo.[5]

2.3 Mixed Reality con pass-through video

I visori MR di ultima generazione integrano telecamere esterne e *pass-through* video, consentendo di sovrapporre contenuti digitali al mondo reale con precisione crescente. Studi recenti di user experience mostrano come la qualità del pass-through influenzi leggibilità, carico di lavoro e cybersickness; in comparazioni su dispositivi attuali, Meta Quest 3 rappresenta un riferimento accessibile per sviluppo e sperimentazione.[6]

[2] Per un'introduzione divulgativa storica: *Wired, "Augmented Reality: OrCam"*, 2013: https://www.wired.com/2013/06/augmented-reality-orcam.

[3] Sito ufficiale Envision Glasses: https://www.letsenvision.com/glasses/home.

[4] Aira, pagina informativa: https://aira.io/. Accesso: 1/09/2025.

[5] Be My Eyes, annuncio *Be My AI*: https://www.bemyeyes.com.

[6] Si veda, ad es., studio comparativo su qualità del pass-through: https://arxiv.org/html/2502.06382v1.

2.4 EchoLens

Il progetto descritto in questa relazione mira a integrare un sistema di comprensione e descrizione del contesto ambientale, trasformando le informazioni visive in un feedback audio facilmente fruibile dall'utente. In questo modo, chi utilizza l'applicazione può ricevere una rappresentazione verbale della scena circostante al posto di doverla percepire visivamente. L'analisi degli oggetti presenti nello spazio è resa possibile dall'impiego di YOLO, un modello di riconoscimento in tempo reale capace di individuare e classificare gli elementi della stanza con elevata precisione.

Capitolo 3

Sviluppo dell'applicazione

L'applicazione EchoLens sfrutta il flusso video proveniente dal visore Oculus Meta Quest 3, acquisendo sequenze di almeno otto frame consecutivi. Questi frame vengono inviati al sistema di elaborazione, dove il modello YOLO (You Only Look Once) si occupa di rilevare e classificare in tempo reale gli oggetti presenti nella scena.

I risultati del riconoscimento vengono quindi trasformati in una descrizione audio contestuale, che fornisce all'utente un feedback immediato e comprensibile sull'ambiente circostante. Questo processo consente di sostituire la percezione visiva con un supporto uditivo naturale, riducendo il carico cognitivo e migliorando l'autonomia di persone ipovedenti.

3.1 Struttura della scena in Unity

La scena principale dell'applicazione, denominata *EchoLens*, è stata progettata secondo un'architettura modulare, finalizzata a integrare in modo flessibile le diverse funzionalità necessarie al funzionamento dell'app. La scena è organizzata attorno a una serie di *GameObject* e *Prefab*, ciascuno con un ruolo specifico e chiaramente definito:

- **[BuildingBlock] Camera Rig**: gestisce la posizione e l'orientamento della telecamera principale, fornendo la prospettiva egocentrica dell'u-

tente. Include componenti per il movimento e la rotazione sincronizzati con il visore, assicurando un tracciamento fluido e realistico.

- **[BuildingBlock] Passthrough**: abilita la modalità *pass-through* del visore, permettendo la visualizzazione dell'ambiente reale con la sovrapposizione di contenuti digitali. Gestisce inoltre la calibrazione e l'adattamento della trasparenza dei layer virtuali.

- **[BuildingBlock] Real Hands**: attiva il tracciamento delle mani, consentendo interazioni naturali tramite gesture. Supporta il riconoscimento di gesture predefinite e personalizzate, integrandosi con i sistemi di input dell'app.

- **Canvas** ed **EventSystem**: costituiscono l'interfaccia grafica utente (*UI*) e gestiscono gli eventi di input, come tocchi, clic e selezioni, permettendo un'interazione coerente tra elementi 2D e 3D.

- **TTS (Text-to-Speech)**: modulo dedicato alla conversione del testo in parlato, utilizzato per descrivere oggetti, azioni o risultati generati dal sistema di riconoscimento. Garantisce una comunicazione accessibile e immediata.

- **Streaming**: componente responsabile della cattura e trasmissione del flusso video verso il sistema di elaborazione, dove modelli di computer vision analizzano i frame in tempo reale. Include funzionalità di compressione e buffering per ottimizzare la latenza.

- **WebCamTextureManagerPrefab**: prefab incaricato della gestione della videocamera, cattura dei frame e loro resa disponibili al sistema di riconoscimento. Supporta configurazioni multiple della videocamera e l'acquisizione simultanea di diversi stream video.

- **[BuildingBlock] Spatial Anchor Core**: nucleo delle funzionalità di ancoraggio spaziale, che consente di posizionare oggetti virtuali persistenti nello spazio 3D e di mantenerne la coerenza durante l'interazione.

- **[BuildingBlock] Controller Buttons Mapper**: gestisce la mappatura dei pulsanti dei controller, permettendo all'utente di interagire anche tramite input fisici, in alternativa o in combinazione alle gesture manuali.

- **[BuildingBlock] Sample Spatial Anchor Controller**: modulo di esempio che fornisce il controllo delle ancore spaziali e funge da riferimento per l'implementazione personalizzata delle funzionalità di ancoraggio.

Questa struttura modulare offre diversi vantaggi:

- **Scalabilità**: è possibile aggiungere nuovi moduli (ad esempio, per l'analisi audio, il riconoscimento vocale o ulteriori modalità di interazione) senza alterare la struttura di base della scena.

- **Manutenibilità**: ogni componente è isolato e facilmente aggiornabile o sostituibile, riducendo il rischio di introdurre errori nella scena complessiva.

- **Riutilizzabilità**: i *Prefab* e i *BuildingBlock* possono essere impiegati in altre scene o progetti, accelerando lo sviluppo di nuove funzionalità.

In sintesi, l'organizzazione della scena in *EchoLens* è progettata per supportare sia l'evoluzione futura dell'applicazione sia un'interazione immersiva, naturale e accessibile per l'utente.

3.2 Architettura di Comunicazione e Streaming Real-Time

L'intero sistema è progettato per consentire la trasmissione in tempo reale dei dati video catturati da un'applicazione Unity verso un server locale Python, la loro analisi automatica e la generazione di descrizioni vocali della scena. Il flusso di comunicazione combina discovery della rete, streaming WebSocket e analisi basata su intelligenza artificiale.

3.2.1 Scoperta Automatica del Server

Per semplificare la configurazione e l'avvio del client Unity, viene implementata una procedura di *Server Discovery* tramite UDP:

- Il client invia un pacchetto broadcast contenente la keyword `DISCOVER_SERVER` sulla porta UDP `8888`.
- Il server Python ascolta sulla stessa porta, intercetta la richiesta e risponde con il proprio indirizzo IP.
- Questa procedura permette al client di connettersi automaticamente al server senza configurazioni manuali, garantendo un setup rapido e affidabile in reti locali.

3.2.2 Streaming dei Frame Video via WebSocket

Una volta individuato il server, il client Unity stabilisce una connessione WebSocket (porta `5001`) e inizia lo streaming dei frame catturati dalla fotocamera:

- I frame vengono acquisiti tramite `WebCamTextureManager`, convertiti in array di byte e memorizzati temporaneamente in memoria.
- La registrazione può essere controllata dall'utente tramite input hardware (ad esempio pulsanti OVR), permettendo l'avvio e l'arresto della cattura.

- I frame vengono inviati in batch al server, con piccoli ritardi tra l'invio di ciascun frame per non saturare la connessione.
- Il client gestisce asincronamente la ricezione dei messaggi dal server, aggiornando la UI e riproducendo eventuali risposte tramite TTS.

3.2.3 Elaborazione dei Frame sul Server Python

Il server riceve i frame dal client e applica un processo di analisi avanzata:

- I frame ricevuti vengono salvati temporaneamente nella cartella `frames/`.
- Dopo aver ricevuto un batch di 8 frame, il server seleziona il frame centrale per l'elaborazione.
- La scena viene analizzata per condizioni generali (luminosità, contrasto, nitidezza) e per il rilevamento degli oggetti con il modello YOLOv8.
- Sulla base dei dati raccolti, il server genera una descrizione testuale e vocale della scena.
- La sintesi vocale è gestita tramite `pyttsx3` in modo asincrono, evitando blocchi dell'event loop.

3.2.4 Flusso Completo di Comunicazione

L'architettura complessiva può essere descritta nei seguenti passaggi:

1. Avvio del server Python, che inizializza il discovery UDP, il server WebSocket e il motore TTS.
2. Il client Unity invia una richiesta UDP e riceve l'indirizzo IP del server.
3. Il client stabilisce la connessione WebSocket e può iniziare a catturare e trasmettere i frame video.
4. Il server riceve i frame, li salva e li elabora per ottenere informazioni sulla scena e sugli oggetti rilevati.

5. Il server invia al client una risposta testuale contenente la descrizione della scena e, se disponibile, la riproduce tramite TTS.

6. Il client aggiorna la UI con il testo ricevuto e riproduce l'audio, fornendo un feedback in tempo reale all'utente.

3.2.5 Caratteristiche e Vantaggi dell'Architettura

- **Rilevamento automatico del server**: semplifica l'installazione e il deployment.
- **Streaming real-time**: batch di frame inviati con bassa latenza.
- **Analisi scena intelligente**: combinazione di rilevamento oggetti e valutazione delle condizioni ambientali.
- **Sintesi vocale asincrona**: feedback immediato all'utente senza blocchi dell'applicazione.
- **Gestione robusta dei frame**: pulizia automatica dei frame temporanei e gestione di eventuali errori nella trasmissione o nell'elaborazione.

Questa combinazione di tecnologie consente un sistema interattivo di visione egocentrica in tempo reale, con supporto per feedback testuali e vocali, ideale per applicazioni immersive basate su Unity e intelligenza artificiale.

3.3 Ancore Spaziali

In questa sezione descrivo l'implementazione delle *Ancore Spaziali* all'interno del progetto sviluppato in Unity, a partire dalla scena `EchoLens`. La *Hierarchy* della scena comprende diversi componenti fondamentali, tra cui:

- `[BuildingBlock] Camera Rig`, `Passthrough` e `Real Hands`, responsabili rispettivamente della gestione della visuale immersiva, della sovrapposizione del mondo reale e del tracciamento delle mani;
- `Streaming` e `TTS`, che gestiscono lo streaming dei frame e il text-to-speech;
- `WebCamTextureManagerPrefab`, utilizzato per la gestione della webcam;
- `[BuildingBlock] Spatial Anchor Core` e `Sample Spatial Anchor Controller`, che costituiscono i moduli centrali per la gestione delle ancore spaziali.

3.3.1 Preparazione dell'ambiente Unity

La configurazione della scena parte quindi da questa struttura, nella quale il Prefab dell'oggetto virtuale è stato aggiunto e collegato allo script `streaming.cs`. Il legame con i moduli `Spatial Anchor Core` e `Spatial Anchor Controller` è particolarmente importante, poiché questi componenti forniscono la base per l'ancoraggio spaziale persistente degli oggetti generati.

3.3.2 Gestione del pinch

Il riconoscimento del pinch avviene attraverso i `Real Hands`, già presenti in Hierarchy. Il loro tracciamento permette di individuare le coordinate 3D delle dita, utilizzate per posizionare le ancore. Lo script `streaming.cs` intercetta il gesto e richiama la funzione di creazione dell'oggetto virtuale.

Esempio per la mano sinistra: [language=C, caption=Gestione del pinch per la mano sinistra] bool isLeftPinching = IsPinching(leftHand); if (isLeft-

Pinching !wasLeftPinching) Debug.Log(" Pinch mano sinistra rilevato - Creazione Ancora"); SpawnAndKeepObject(leftHand.transform.position); wasLeftPinching = isLeftPinching;

3.3.3 Creazione e mantenimento degli oggetti virtuali

Integrazione con il modulo `Spatial Anchor Core` consente di mantenere gli oggetti creati nello spazio 3D, senza che vengano distrutti al successivo pinch. Il metodo implementato è stato modificato per consentire l'istanziazione permanente di più oggetti:

[language=C, caption=Metodo per la creazione e mantenimento degli oggetti virtuali] private void SpawnAndKeepObject(Vector3 pinchPosition) if (virtualObjectPrefab != null) Instantiate(virtualObjectPrefab, pinchPosition, Quaternion.identity); Debug.Log(" Oggetto virtuale creato al pinch e mantenuto!"); else Debug.LogWarning(" Prefab dell'oggetto virtuale non assegnato!");

3.3.4 Risultati

Con questa implementazione:

- Il tracciamento delle mani (`Real Hands`) fornisce le posizioni di pinch;
- Lo script `Streaming` gestisce la logica di spawn degli oggetti virtuali;
- I moduli `Spatial Anchor Core` e `Sample Spatial Anchor Controller` permettono alle ancore di essere persistenti nello spazio aumentato;
- Ogni pinch corrisponde alla creazione di un'ancora spaziale aggiuntiva, che rimane fissata nella scena senza sostituire le precedenti.

Questo consente all'utente di costruire progressivamente uno spazio popolato da molteplici oggetti virtuali ancorati in posizioni differenti.

3.3.5 Creazione e mantenimento degli oggetti virtuali

La funzione inizialmente progettata prevedeva la distruzione dell'oggetto precedente prima di crearne uno nuovo. Per realizzare invece un sistema di *ancore permanenti*, ho modificato il metodo in modo da non eliminare più gli oggetti istanziati, consentendo così di popolare la scena con più ancore fissate ai pinch.

[language=C, caption=Metodo per la creazione e mantenimento degli oggetti virtuali] private void SpawnAndKeepObject(Vector3 pinchPosition) if (virtualObjectPrefab != null) Instantiate(virtualObjectPrefab, pinchPosition, Quaternion.identity); Debug.Log(" Oggetto virtuale creato al pinch e mantenuto!"); else Debug.LogWarning(" Prefab dell'oggetto virtuale non assegnato!");

3.3.6 Risultati

Grazie a queste modifiche, ogni volta che l'utente esegue un pinch:

- viene rilevato l'evento dalla mano corrispondente;
- viene istanziata una nuova ancora spaziale in corrispondenza della posizione del pinch;
- le ancore precedentemente create restano permanenti in scena, consentendo la costruzione progressiva di un ambiente popolato da più oggetti virtuali.

Questo approccio permette di sfruttare le gesture come meccanismo di interazione naturale con l'ambiente aumentato, trasformando il gesto del pinch in un'operazione di ancoraggio spaziale.

Capitolo 4

Conclusioni

L'applicazione *EchoLens* rappresenta un primo passo concreto verso la creazione di strumenti digitali pensati per persone ipovedenti. Pur essendo progettata principalmente per fornire supporto a utenti con ridotta capacità visiva, mantiene comunque un'interfaccia visiva. Questa interfaccia è volutamente minimalista, progettata per agevolare l'utente e ridurre il rischio di confusione che potrebbe derivare dall'utilizzo di layout complessi o troppo ricchi di elementi.

Attualmente, l'applicazione è pienamente funzionante e consente di riconoscere oggetti, gestire flussi video in tempo reale e fornire descrizioni tramite sintesi vocale, rappresentando un aiuto concreto nel rendere "visibile" un mondo che altrimenti rimarrebbe difficile da percepire per chi ha problemi di vista.

In una versione futura e più completa, potrebbero essere introdotte ulteriori funzionalità di accessibilità, come una guida vocale completa che accompagni l'utente passo passo nell'utilizzo dell'applicazione: dalla modifica delle impostazioni all'uscita dall'app, fino all'interazione con ogni singolo modulo. Tali migliorie renderebbero *EchoLens* ancora più inclusiva, permettendo un'esperienza più intuitiva, autonoma e sicura per le persone ipovedenti.

In conclusione, *EchoLens* si propone come uno strumento versatile e modulare, con il potenziale di migliorare significativamente la percezione dell'ambiente circostante per chi ha una vista limitata, offrendo supporto

concreto attraverso tecnologia immersiva, interattiva e accessibile.

Bibliografia

[1] OrCam Technologies. *OrCam MyEye 2 Pro*. Disponibile su: https://www.orcam.com/en-us/orcam-myeye-2-pro.

[2] Envision. *Envision Glasses*. Disponibile su: https://www.letsenvision.com/glasses/home.

[3] Aira. *Aira Visual Interpreting*. Disponibile su: https://aira.io/.

[4] Be My Eyes. *Introducing Be My AI*. Disponibile su: https://tinyurl.com/bemy-ai-news.

[5] Cugno et al. *Comparing Pass-Through Quality of Mixed Reality Devices*, 2025 (preprint). Disponibile su: https://arxiv.org/html/2502.06382v1.

[6] University of Washington News. *Researchers aim to improve accessibility with AR/VR*, 2023. Disponibile su: https://www.washington.edu/news/2023/10/17/accessibility-augmented-virtual-reality/.